

Тестовое задание Python Developer (Intern / Junior)

Во время работы над тестовым заданием просим вас не использовать инструменты LLM, мы хотим оценить ваши умения а не возможности claude.

Общее описание

Необходимо реализовать три сервиса:

- Клиентский сервис
- Web API сервис
- Сервис фоновой обработки

Все сервисы должны запускаться через Docker Compose одной командой.

Код решения разместить в любом публичном репозитории (GitHub, GitLab, Bitbucket и т.д.).

1. Web API сервис

Необходимо реализовать асинхронный сервис на FastAPI.

Сервис принимает строку следующего формата:

```
{IP адрес} {HTTP method} {URI} {HTTP status code}
```

Пример:

```
192.168.1.1 GET /api/users 200
```

Сервис должен:

- распарсить строку;
- сохранить данные в PostgreSQL;
- присвоить записи уникальный идентификатор;
- сохранить время создания записи;
- предоставлять API для получения сохранённых данных.

Технологии

- Python 3.11+
- FastAPI
- Pydantic
- PostgreSQL

Endpoint'ы

POST /api/data

Request:

```
{
  "log": "192.168.1.1 GET /api/users 200"
}
```

Response:

Status	Описание
201	Лог успешно сохранён
400	Некорректный формат данных
500	Внутренняя ошибка сервиса

GET /api/data

Параметры запроса необходимо спроектировать самостоятельно.

Сервис должен возвращать ранее сохранённые записи.

```
[
  {
    "id": "uuid",
    "created": "2025-01-01T12:00:00",
    "log": {
      "ip": "192.168.1.1",
      "method": "GET",
      "uri": "/api/users",
      "status_code": 200
    }
  }
]
```

2. Клиентский сервис

Сервис должен генерировать строки формата:

```
{IP адрес} {HTTP method} {URI} {HTTP status code}
```

и отправлять их в Web API сервис через POST-запросы.

Требования:

- работа в N потоков или процессов;
- случайная задержка между запросами до M миллисекунд;
- значения N и M должны задаваться через переменные окружения;
- все отправленные сообщения должны логироваться в файл.

3. Сервис фоновой обработки

Сервис должен периодически выполнять GET-запросы к Web API сервису и сохранять полученные данные в файл, примонтированный как Docker Volume.

Требования:

- запуск по таймеру;
- сохранение результатов в общий файл;
- учитывать возможность запуска нескольких экземпляров сервиса одновременно с использованием одного разделяемого файла.

Требования к проекту

- Использовать Docker для каждого сервиса.
- Запуск всей системы должен выполняться через docker-compose.
- Конфигурация сервисов должна выполняться через переменные окружения.
- Соблюдать читаемую структуру проекта.
- Использовать типизацию Python.

README

- Краткое описание архитектуры решения.
- Инструкцию по запуску проекта.
- Описание используемых технологий.
- Перечень переменных окружения.
- Описание принятых архитектурных решений и допущений.
- Примеры запросов к API.

Задание со звездочкой (не обязательно)

GET /api/stats

```
{
  "methods": {
    "GET": 120,
    "POST": 95,
    "PUT": 12
  },
  "status_codes": {
    "200": 210,
    "404": 15,
    "500": 2
  }
}
```

- данные должны вычисляться на основе записей, сохранённых в базе данных;
- использовать агрегирующие SQL-запросы;
- решение должно эффективно работать на большом количестве записей.

Что будет оцениваться

- Качество кода.
- Архитектура решения.
- Работа с FastAPI и PostgreSQL.
- Использование Docker.
- Работа с конкурентностью и многопоточностью.
- Умение документировать решение.
- Обработка ошибок и граничных случаев.